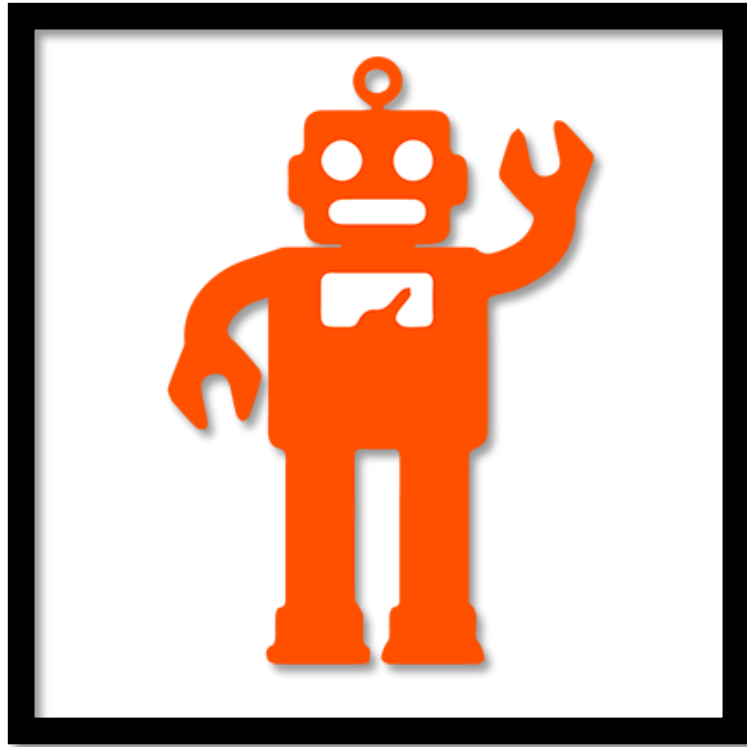




Sabbir Raihan Shishir

Robo Garage Uprising

*Physical Robot Soccer with Computer Vision
and Reinforcement Learning*

ML-Agents Setup & Installation Guide

Contents

1	Introduction	1
2	Step-by-Step Installation	1
2.1	Install Microsoft Visual C++ Redistribute	1
2.2	Install NVIDIA CUDA Toolkit (GPU users only)	2
2.3	Install cuDNN (GPU users only)	2
2.4	Install Miniconda	2
2.5	Create the ML-Agents Python Environment	3
2.6	Install ML-Agents Python Package from Source	4
3	Unity Side Setup	4
3.1	Download the Unity Project	4
3.2	Open the Project in Unity	5
3.3	Install the ML-Agents Unity Package	5
4	Training the ML Agent	6
4.1	Pre-flight Checklist	6
4.2	Start Training	7
4.3	Monitor with TensorBoard	7
4.4	Stop Training	8
4.5	Tips	8
5	Deploying the Trained AI to the Physical Robot	8
5.1	Download Required Files	8
5.2	Physical Arena & Camera Setup	9
5.3	Run the Brain	9

1 Introduction

This guide explains how to set up the Unity ML-Agents environment for training AI agents in the Robo Garage Uprising simulation. Follow the steps in order to get from clean machine to a running training session.

The setup includes:

- Miniconda - creates an isolated Python environment so ML-Agents dependencies do not interfere with other projects on your system.
- NVIDIA CUDA and CuDNN toolkit - optional for GPU accelerated training only
- ML-Agents Python package - the command-line trainer
- ML-Agents Unity package - the in-editor agent scaffolding

This environment will be used later to train the AI inside Unity.

Note: You do not need a powerful GPU to run ML-Agents. CPU training works fine, though it is slower. GPU acceleration is only recommended if you want faster training.

2 Step-by-Step Installation

2.1 Install Microsoft Visual C++ Redistribute

This runtime library is required by several Python packages on Windows.

1. Go to the Microsoft download page:

<https://learn.microsoft.com/en-us/cpp/windows/latest-supported-vc-redist>

2. Click the x64 link and download the latest version.
3. Run the installer and follow the on-screen instructions.

2.2 Install NVIDIA CUDA Toolkit (GPU users only)

Skip this section and section 2.3 if you plan to train on CPU only.

1. Check your GPU compute capability at:
<https://developer.nvidia.com/cuda/gpus>
2. Download the compatible CUDA Toolkit from:
<https://developer.nvidia.com/cuda-toolkit-archive>
3. Run the installer and choose the Express option.

2.3 Install cuDNN (GPU users only)

1. Download cuDNN from:
<https://developer.nvidia.com/cudnn-downloads>
2. Download the version that matches your installed CUDA Toolkit.
3. Check the CUDA / cuDNN compatibility matrix at:
<https://docs.nvidia.com/deeplearning/cudnn/backend/latest/reference/support-matrix.html>
4. Install following NVIDIA's instructions.

2.4 Install Miniconda

1. Download Miniconda from:
<https://www.anaconda.com/download>
2. Run the installer and check both of these options during setup:
 - ✓ Create shortcuts
 - ✓ Clear the package cache upon completion

2.5 Create the ML-Agents Python Environment

- 1 Open Anaconda Powershell Prompt (search in Windows Start Menu)

- 2 Create a new environment with Python 3.10.12:

```
conda create -n mlagents python=3.10.12
```

Type - y and press Enter when asked to proceed.

- 3 Activate the environment:

```
conda activate mlagents
```

- 4 Install a compatible version of NumPy:

```
conda install numpy=1.23.5
```

- 5 Install PyTorch

For CPU Only:

```
pip3 install torch~=2.2.1 --index-url https://download.pytorch.org/whl/cpu
```

For GPU Support (CUDA 12.1):

```
pip3 install torch~=2.2.1 --index-url https://download.pytorch.org/whl/cu121
```

- 6 Verify the installation:

```
python

>>> import torch

>>> import numpy

>>> print("PyTorch:", torch.__version__)

>>> print("NumPy:", numpy.__version__)

>>> exit()
```

You should see the versions printed without any errors.

- 7 (Optional) Clear the terminal screen:

```
clear
```

2.6 Install ML-Agents Python Package from Source

- 1 Download the latest ML-Agents release from GitHub:
<https://github.com/Unity-Technologies/ml-agents/releases>
- 2 Click on the latest release → scroll down and download the Source code (zip) file.
- 3 Extract the zip file into the folder where you keep your Unity projects.
- 4 Rename the extracted folder to ml-agents (so the full path is easy to find).
- 5 In Anaconda PowerShell Prompt, change directory to the ml-agents folder:

```
cd path\to\your\ml-agents
```

(Replace path\to\your\ml-agents with the actual location.)

- 6 Install the ML-Agents packages:

```
python -m pip install ./ml-agents-envs  
python -m pip install ./ml-agents
```

- 7 Verify the installation:

```
mlagents-learn --help
```

If you see the help text (a list of options) without errors, the Python side is ready.

3 Unity Side Setup

3.1 Download the Unity Project

- 1 Go to the RoboGarage - RobotUprising repository.
- 2 Click the green **<> Code button -> Download ZIP**.
- 3 Extract the ZIP file to a convenient location on your computer.

Note: Do not move or rename folders inside the project after extracting, or Unity may lose references

3.2 Open the Project in Unity

- 1 Open Unity Hub.
- 2 Click Add project from disk.
- 3 Navigate to the extracted folder and select the folder that contains the Assets, Packages, and ProjectSettings folders.
- 4 Click Open and wait for Unity to load the project.

Once the project opens, you should see a ready scene (Make sure you are in the **TrainingArena** scene which you will find it in the folder called Scenes)

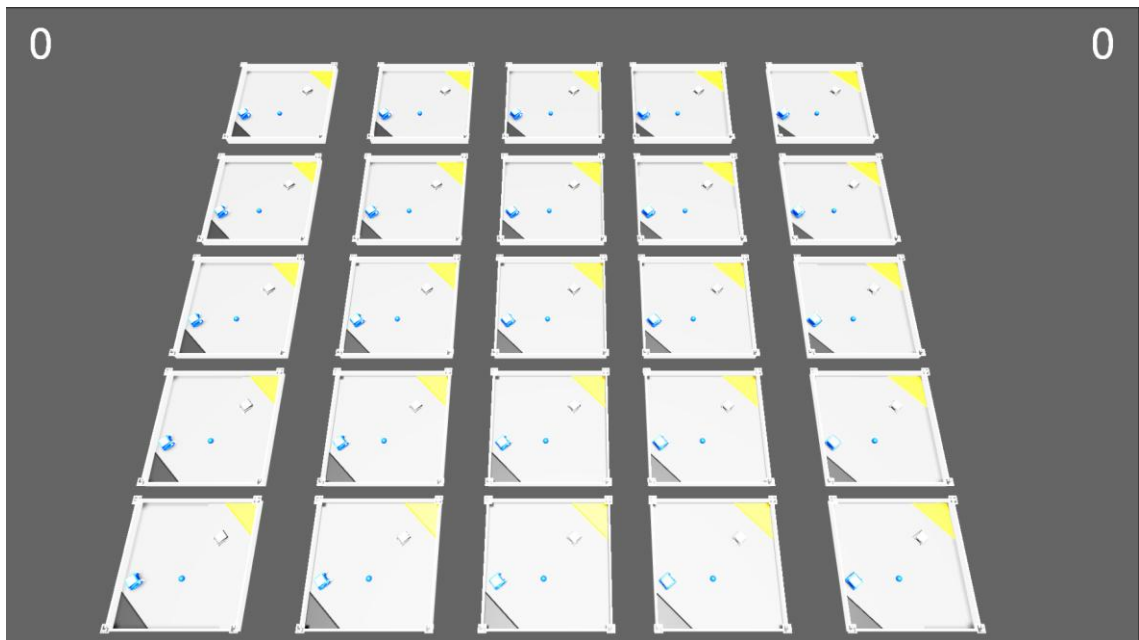


Figure 1: TrainingArena Scene

3.3 Install the ML-Agents Unity Package

- 1 In Unity, go to the top menu: **Window -> Package Manager**.
- 2 Set the source to Packages: Unity Registry.
- 3 Search for “ML Agents” and install the latest version that matches your Python ML - Agents release. (Version 4.0.2 is recommended)

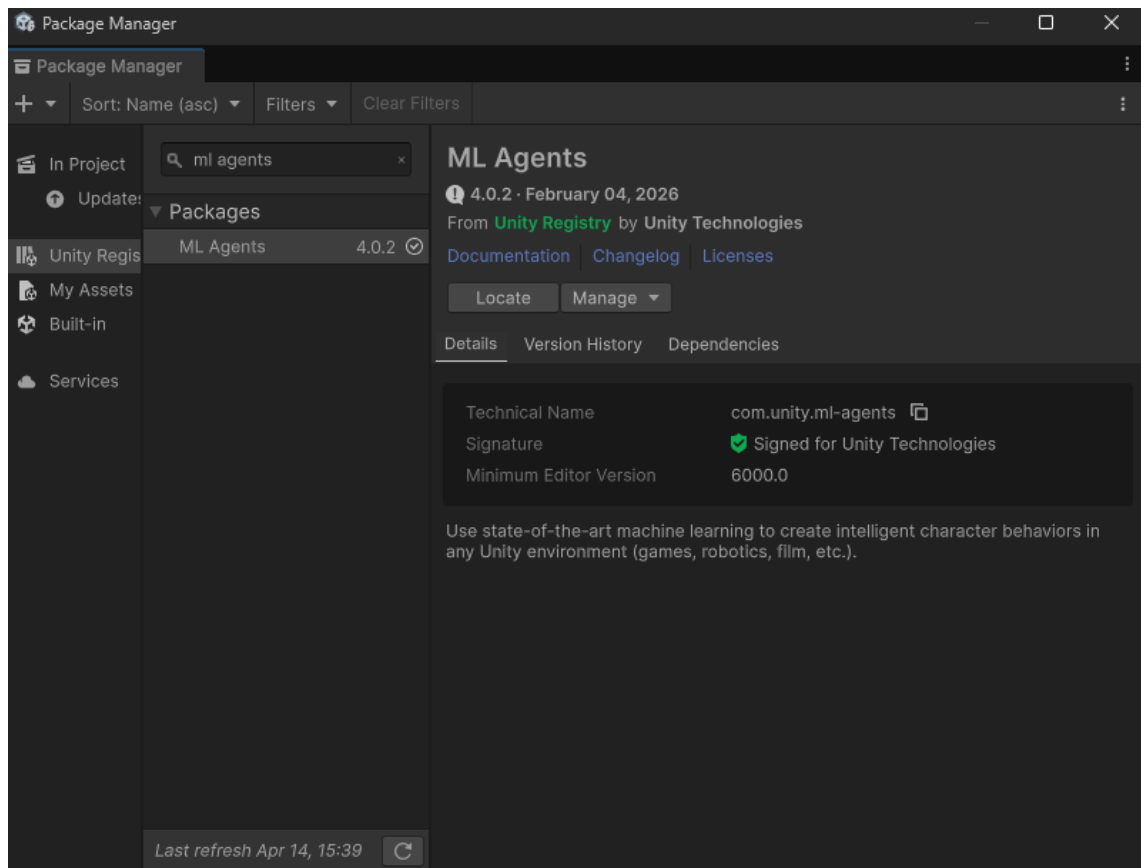


Figure 2: ML Agents package in Package Manager

After installation, you should see ML Agents listed under In Project in the Package Manager

4 Training the ML Agent

After completing Unity setup and the Python ↔ Unity synchronization (Phase 2), you can start training the AI for the robots.

Step-by-Step Training Instructions

4.1 Pre-flight Checklist

1. Training arena scene is open with blue robots having Behavior Parameters + Decision Requester + RobotAgent script.

2. Training config file exists at Assets/Config.

4.2 Start Training

1. Open Anaconda Powershell Prompt, and run:

```
conda activate mlagents
```

2. Navigate to your Unity project folder.

3. Start training with this command:

```
mlagents-learn Config/config file name.yaml --run-id=RobotUprising
```

4. Resume training with this command:

```
mlagents-learn Config/config file name.yaml --run-id=RobotUprising --resume
```

5. Overwrite training with this command:

```
mlagents-learn Config/config file name.yaml --run-id=RobotUprising --force
```

4.3 Monitor with TensorBoard

1. Unity Editor must stay open and the scene must be in **Play Mode**.
2. Training will start automatically. You will see progress in the Anaconda terminal.
3. For better monitoring, open **TensorBoard** in second Anaconda window and write this command:

```
tensorboard --logdir results
```

Then open <http://localhost:6006> in your browser. A positive upward trend in the reward curve means the agent is learning correctly.

4.4 Stop Training

- Press **Ctrl + C** in the Anaconda window when you are satisfied with the results (usually after 500,000 – 2,000,000 steps).

4.5 Tips

1. First training can take from 30 minutes to several hours depending on your computer.
2. Start with fewer steps (max_steps: 200000 in YAML) to test if everything works.
3. Watch the rewards in TensorBoard. Positive trend = AI is learning.
4. If the reward curve is flat or declining after many steps, review the observations and reward shaping in RobotAgent.cs.

5 Deploying the Trained AI to the Physical Robot

After training is complete, use brain_runner.py to drive a real ESP32 robot. The script reads the overhead camera, detects the robot and ball with computer vision, and sends motor commands via UDP.

5.1 Download Required Files

1. Download the brain_runner.py and config.py from the Robot Uprising GitHub repository.
2. Copy your trained .onnx brain file from (results/<latest-run-folder>) into the same folder where brain_runner.py is located.

5.2 Physical Arena & Camera Setup

1. Mount the webcam directly overhead and centred over the 1.5m x 1.5m arena.
2. Attach ArUco markers to the robots and goals as specified in the project documentation.
3. Place the four corner markers (IDs 46-49) at the arena corners. All four must be visible for automatic calibration.
4. Connect the ESP32 robot to the same WiFi network as your computer, the update ROBOT1_IP in config.py.

5.3 Run the Brain

```
cd path/to/brain_runner_folder  
python brain_runner.py
```

Then press Play in Unity. Unity sends a START message automatically once the camera initialises. The connection dot in the HUD turns green when heartbeats are received from Python.

Note: brain_runner.py requires opencv-python, numpy and onnxruntime. Install them with: `pip install opencv-python numpy onnxruntime`

